Jr.Pen 13.2. Linux Privilege Escalation: Enumeration

Task 1 - Introduction

Privilege escalation is a context-dependent, step-by-step process with no single shortcut to root access. Privilege escalation is the unauthorized gain of higher-level access by exploiting system vulnerabilities or misconfigurations. Privilege escalation is crucial because it lets you gain system administrator levels of access, which allows you to perform actions such as:

- Resetting passwords
- Bypassing access controls to compromise protected data
- Editing software configurations
- Enabling persistence
- Changing the privilege of existing (or new) users
- Execute any administrative command

Task 2 - What Is Enumeration?

Enumeration is the systematic gathering of information about a target system to discover security flaws, misconfigurations, or credentials needed for privilege escalation. Enumeration is the vital discovery phase that turns blind guesswork into targeted, effective privilege escalation.

Task 3 - OS Enumeration

The `hostname` Command will return the hostname of the target machine.

The `uname -a` Command will print system information, giving you additional details about the kernel used by the system.

```
john@home:~$ uname -a
Linux home 6.8.0-41-generic
```

In the example above, the output is as follows:

- **Linux**: Operating system
- **home**: Hostname
- **6.8.0-41-generic**: Kernel version

The `/proc/version` File provides information about the target system processes. You will find proc on many different Linux flavors, making it an essential tool to have in your arsenal.

```
john@home:~$ cat /proc/version
Linux version 6.8.0-41-generic (buildd@lcy02-amd64-077) (x86_
64-linux-gnu-gcc-13 (Ubuntu 13.2.0-23ubuntu4) 13.2.0, GNU ld
(GNU Binutils for Ubuntu) 2.42)
```

In the example above, the output is as follows:

- **6.8.0-41-generic**: Kernel version
- **buildd@lcy02-amd64-077**: Build machine that compiled the kernel
- **x86_64-linux-gnu-gcc-13**: Compiler used to build the kernel
- **Ubuntu 13.2.0-23ubuntu4**: Ubuntu package version of the gcc compiler
- **GNU ld (GNU Binutils for Ubuntu) 2.42**: Linker used during the kernel build

The `/etc/issue` File: This file usually contains some information about the operating system, but can easily be customized or changed. Also, any file containing system information can be customized or changed.

```
john@linux-enumeration:~$ cat /etc/issue
Ubuntu 22.04.1 LTS \n \l
```

In the example above, `Ubuntu 22.04.1 LTS` represents the Ubuntu version running on the host.

The `ps` Command is an effective way to see the running processes on a Linux system. Running `ps` in your terminal will show processes for the current shell. The `ps` command provides a few useful options.

- `ps aux`: The `aux` option will show processes for all users `a`, display the user that launched the process `u`, and show processes that are not attached to a terminal `x`. Looking at the `ps aux` command output, you can have a better understanding of the system and potential vulnerabilities.
- `ps axjf`: The `axjf` option will show processes from all users `a`, include processes that have no controlling terminal `x`, use jobs format output `j`, and forest/tree view `f`. An example of such tree can be seen below.

The `cron` Service: Cron is a time-based job scheduler in Linux that allows users and the system to schedule commands or scripts to run automatically at specified intervals. You can start by examining `/etc/crontab`

The `dpkg` Command allows us to query all installed packages and their version. The command to list all installed software is `dpkg -l`.

Answer the questions below

> What is the hostname of the target host?

linux-enumeration

run command `hostname`

> What is the Linux kernel version of the target host?

6.8.0-1017-aws

```
john@linux-enumeration:~$ uname -a
Linux linux-enumeration 6.8.0-1017-aws #18-Ubuntu SMP Wed Oct
20:17:03 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux
```

> What version of Ubuntu is running on the host?

Ubuntu 24.04.1 LTS

run the command `cat /proc/version`

> What is the full path of the script run by root every 5 minutes?

/root/backup.sh

run the command `cat /etc/issue`

> What is the full version of AppArmor?

4.0.1really4.0.1-0ubuntu0.24.04.3

run the command `dpkg -s apparmor | grep Version`

Task 4 - User Enumeration

The `id` Command will provide a general overview of the user's privilege level and group memberships.

```
john@home:~$ id
uid=1001(john) gid=1001(john) groups=1001(john),100(users)
john@home:~$ id matt
uid=1002(matt) gid=1002(matt) groups=1002(matt),27(sudo),116
(admin)
```

The `env` Command will show environmental variables.

```
john@home:~$ env
MAIL=/var/mail/john
USER=john
SSH_CLIENT=10.80.88.217 50500 22
HOME=/home/john
```

The `history` Command can give us some idea about the target system and, albeit rarely, store information such as passwords or usernames.

The `sudo -l` Command can be used to list all commands your user can run using sudo.

The `/etc/passwd` File can be an easy way to discover users on the system.

```
john@home:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
```

It is a shell pipeline that extracts and lists all usernames on the system from `/etc/passwd`.

```
john@home:~$ cat /etc/passwd | cut -d ":" -f 1
root
lp
mail
```

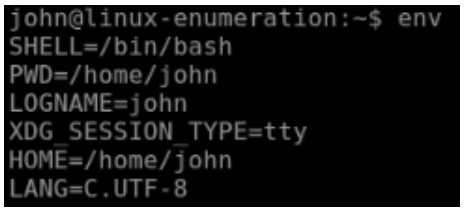
It is a command that filters and lists all user accounts on the system that have a home directory located in `/home`.

```
john@home:~$ cat /etc/passwd | grep /home
matt:x:1000:1000:matt,,,:/home/matt:/bin/bash
karen:x:1001:1001::/home/karen:
```

Answer the questions below

> Print the environment variables. What is the value of **LANG**?

C.UTF-8



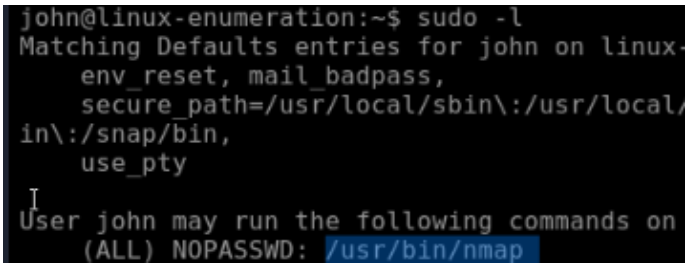
> What is the flag in your history?

THM{history-is-not-safe}



> What is the full path of the command you are allowed to run with elevated privileges?

/usr/bin/nmap



> What is the username of the Mailing List Manager?

list



Task 5 - Network Enumeration

The `ifconfig` Command gives you information about the network interfaces of the system. I

The `netstat` Command can be used with several different options to gather information on existing connections.

`netstat -a` shows all listening ports and established connections.

`netstat -at` or `netstat -au` can also be used to list TCP or UDP protocols, respectively.

`netstat -l` lists ports in “listening” mode. These ports are open and ready to accept incoming connections. This can be used with the `-t` option to list only ports that are listening using the TCP protocol,

`netstat -s` lists network usage statistics by protocol. This can also be used with the `-t` or `-u` options to limit the output to a specific protocol.

`netstat -tp` lists connections with the service name and PID information. This can also be used with the `-l` option to list listening ports/services. The `-n` flag tells netstat to show numeric addresses and port numbers instead of resolving them.

`netstat -i` shows interface statistics.

`netstat -ano`The netstat usage you will probably see most often in blog posts, write-ups, and courses is `netstat -ano` which could be broken down as follows;

- `a`: Display all sockets

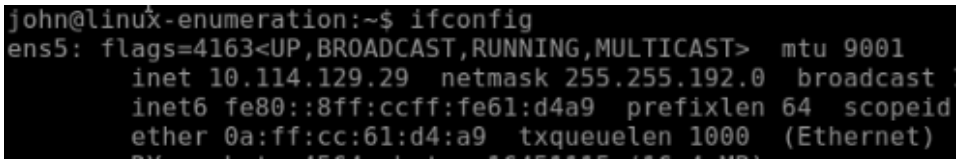
- `n`: Do not resolve names
- `o`: Display timers

Feature	Netcat (nc)	Netstat (netstat)
Primary Purpose	**Active Networking:** Reads and writes data across TCP or UDP network connections.	**System Inspection:** Displays active network connections, open ports, and routing tables.
Common Uses	Port scanning, banner grabbing, file transfers, remote shells, listening on ports.	Enumerating open ports, auditing active connections, checking process IDs (PIDs).
Role in Enumeration	Used to connect to service ports or catch reverse shells.	Used to discover local listening services (e.g., internal-only web servers).

Answer the questions below

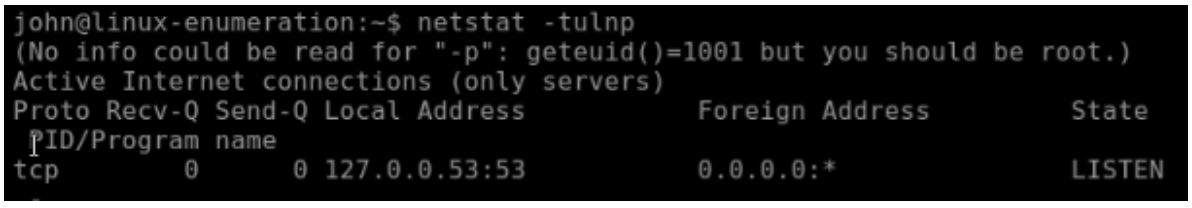
> What is the name of the network interface, other than loopback?

ens5



> What port, other than 22, is listening on the host?

53



Task 6 - File Enumeration

The `ls` Command use potential privilege escalation vectors, remember to always use the `ls` command with the `-la` parameter.

The `find` Command searches the target system for important information and potential privilege escalation vectors can be fruitful.

- `find . -name flag1.txt` finds the file named “flag1.txt” in the current directory and subdirectories recursively
- `find /home -name flag1.txt` finds the file names “flag1.txt” in the /home directory and subdirectories recursively
- `find / -type d -name config` finds the directory named config under “/”
- `find / -type f -perm 0777` finds files with the 777 permissions (files readable, writable, and executable by all users)
- `find / -perm -a=x` finds executable files
- `find /home -user frank` finds all files for user “frank” under “/home”
- `find / -mtime -10` finds files that were modified in the last 10 days
- `find / -atime -10` finds files that were accessed in the last 10 days
- `find / -cmin -60` finds files changed within the last hour (60 minutes)
- `find / -amin -60` finds files accessed within the last hour (60 minutes)
- `find / -size +50M` finds files with at least 50 MB size

The example above returns files that are larger than 100 MB. It is important to note that the `find` command tends to generate errors, which sometimes makes the output hard to read. To address this, we can use `2>/dev/null`, which sends errors to `/dev/null`, effectively ignoring them.

Folders and files that can be written to or executed from:

- `find / -writable -type d 2>/dev/null` finds world-writeable folders.
- `find / -perm -222 -type d 2>/dev/null` finds world-writeable folders.
- `find / -perm -o w -type d 2>/dev/null` finds world-writeable folders.
- `find / -perm -o x -type d 2>/dev/null` finds world-executable folders

Find development tools and supported languages:

- `find / -name perl*`
- `find / -name python*`

- `find / -name gcc*`

Wildcards can be used when you don't know the exact name of a file you are looking for. Say, for example, you want to find a text file containing passwords. You could use the following command.

- `find / -name pass*.txt`

The command above will find file names such as `pass.txt`, `password.txt`, `passwords.txt`, etc.

Below is a short example used to find files that have the SUID bit set. The SUID bit allows the file to run with the privilege level of the account that owns it, rather than the account that runs it.

- `find / -perm -u=s -type f 2>/dev/null` finds files with the SUID bit, which allows us to run the file with a higher privilege level than the current user.

Answer the questions below

> What are the contents of the secret file in your home folder?

THM{not-so-hidden}

john@linux-enumeration:~$ ls -la cat hidden_secret.txt

> Find a file that has TryHackMe in its name. What is its content?

THM{found-the-flag}

john@linux-enumeration:~$ find / -type f -name "*TryHackMe*" 2>/dev/null
/var/local/TryHackMe-flag.txt

john@linux-enumeration:~$ cat /var/local/TryHackMe-flag.txt
THM{found-the-flag}

Task 7 - Conclusion

In this room, you've learned:

- How to find information about the system you are targeting
- How to enumerate local users

- How to leverage `ifconfig` and `netstat` to gain information on network connections
- How to find interesting or hidden files on Linux systems

As you could see, there is a lot of information to be gained about a target system that doesn't necessarily give you a clear-cut escalation path, but does aid in your privilege escalation efforts. Please spend some time getting comfortable with commands such as `find`, `locate`, `grep`, `cut`, `sort`, etc.